

Lekce 3: Řídicí struktury a funkce v JavaScriptu

1. Osnova obsahu

A. Úvod do řídicích struktur

- **Co jsou řídicí struktury?**
 - Mechanismy, které řídí tok vykonávání v programu.
 - Umožňují vývojářům určovat pořadí, v jakém se kód vykonává na základě podmínek nebo iterací.
- **Typy řídicích struktur:**
 - **Podmíněné příkazy:** `if` , `else if` , `else` , `switch` .
 - **Cyklus:** `for` , `while` , `do...while` , `for...of` , `for...in` .

Co jsou řídicí struktury?

Definice:

Řídicí struktury jsou programovací konstrukce, které určují tok vykonávání v rámci programu.

Rozhodují o tom, v jakém pořadí se vykonávají jednotlivé příkazy, což umožňuje vývojářům realizovat rozhodování a opakující se úlohy v jejich kódu.

Podrobné vysvětlení:

- **Účel:** Řídicí struktury umožňují tvorbu dynamických a flexibilních programů tím, že umožňují kódu reagovat různě na základě různých podmínek nebo opakovat určité operace vícekrát.
- **Typy řídicích struktur:**
 - Podmíněné příkazy:** Spouští různé bloky kódu v závislosti na tom, zda je podmínka splněna, nebo ne.
 - Cykly:** Opakují blok kódu tak dlouho, dokud daná podmínka zůstává splněná.
- **Důležitost v programování:**
 - **Rozhodování:** Umožňuje programům dělat rozhodnutí a vybírat, který kód vykonat.
 - **Opakování:** Usnadňuje provádění opakovaných úloh bez zbytečného opakování kódu.
 - **Efektivita:** Zlepšuje efektivitu a čitelnost kódu tím, že snižuje duplicitní kód a stručně řeší složitou logiku.

Vizuální znázornění:

- **Vývojové diagramy:** Často se používají pro vizualizaci řídicích struktur, zobrazující tok vykonávání na základě podmínek a iterací.

B. Podmíněné příkazy

- **Příkaz `if` :**
 - Spouští blok kódu, pokud je daná podmínka splněna.
 - Syntaxe a základní použití.
 - Příkladové scénáře v testovací automatizaci.
- **Příkazy `else if` a `else` :**
 - Umožňují přidat další podmínky a alternativy.
 - Rozšiřují možnosti rozhodování ve skriptech.
- **Příkaz `switch` :**
 - Vyhodnotí výraz podle více případů.
 - Zjednodušuje opakované podmíněné kontroly.

Podmíněné příkazy

Definice:

Podmíněné příkazy jsou řídicí struktury, které vykonávají specifické bloky kódu podle toho, zda je daná podmínka pravdivá, nebo nepravdivá.

Podrobné vysvětlení:

- **Typy podmíněných příkazů:**

i. Příkaz `if` :

- **Účel:** Spouští blok kódu, pokud je daná podmínka splněna.
- **Syntaxe:**

```
if (podmínka) {  
    // kód, který se spustí, když je podmínka pravdivá  
}
```

- **Příklad:**

```
let score = 85;  
if (score >= 80) {  
    console.log("Skvělá práce!");  
}
```

ii. Příkaz `else if` :

- **Účel:** Umožňuje přidat další podmínky, pokud předchozí `if` nebyla pravdivá.

- **Syntaxe:**

```
if (podmínka1) {  
    // kód, když je podmínka1 pravdivá  
} else if (podmínka2) {  
    // kód, když je podmínka2 pravdivá  
}
```

- **Příklad:**

```
let score = 75;  
if (score >= 90) {  
    console.log("Výborně!");  
} else if (score >= 80) {  
    console.log("Skvělá práce!");  
}
```

iii. Příkaz else :

- **Účel:** Spouští blok kódu, pokud žádná z předchozích podmínek nebyla splněna.

- **Syntaxe:**

```
if (podmínka1) {  
    // kód, když je podmínka1 pravdivá  
} else if (podmínka2) {  
    // kód, když je podmínka2 pravdivá  
} else {  
    // kód, když žádná podmínka není pravdivá  
}
```

- **Příklad:**

```
let score = 55;  
if (score >= 90) {  
    console.log("Výborně!");  
} else if (score >= 80) {  
    console.log("Skvělá práce!");  
} else {  
    console.log("Jen tak dál!");  
}
```

iv. Příkaz switch :

- **Účel:** Vyhodnotí výraz podle více hodnot a spustí odpovídající blok kódu.

- **Syntaxe:**

```

switch (výraz) {
  case hodnota1:
    // kód, když výraz === hodnota1
    break;
  case hodnota2:
    // kód, když výraz === hodnota2
    break;
  default:
    // kód, když žádný případ neodpovídá
}

```

◦ **Příklad:**

```

let day = "Monday";
switch (day) {
  case "Monday":
    console.log("Začátek pracovního týdne!");
    break;
  case "Friday":
    console.log("Konec pracovního týdne!");
    break;
  default:
    console.log("Střed týdne.");
}

```

Použití v testovací automatizaci:

- **Dynamické chování testů:** Úprava testovacích kroků podle stavu aplikace nebo vstupů uživatele.
- **Ošetření chyb:** Spouštění různých akcí podle toho, zda testy prošly nebo selhaly za určitých podmínek.

C. Cykly

- **Cyklus for :**
 - Iteruje pevně stanovený počet opakování.
 - Syntaxe a příklady použití.
- **Cyklus while :**
 - Spouští se, dokud je podmínka pravdivá.
 - Příklady z testovací automatizace.
- **Cyklus do...while :**
 - Podobné jako while , zaručuje alespoň jedno provedení.

- **Cykly for...of a for...in :**
 - Iterace přes iterovatelné objekty (pole, řetězce) a vlastnosti objektů.

Cykly

Definice:

Cykly jsou řídicí struktury, které opakovaně vykonávají blok kódu, dokud je splněna podmínka.

Podrobné vysvětlení:

- **Typy cyklů:**

- i. **Cyklus for :**

- **Účel:** Spustí blok kódu předem daný počet opakování.
 - **Syntaxe:**

```
for (inicializace; podmínka; přírůstek) {  
    // kód k vykonání  
}
```

- **Příklad:**

```
for (let i = 0; i < 5; i++) {  
    console.log("Iterace:", i);  
}
```

- ii. **Cyklus while :**

- **Účel:** Spouští blok kódu, dokud je podmínka pravdivá.
 - **Syntaxe:**

```
while (podmínka) {  
    // kód k vykonání  
}
```

- **Příklad:**

```
let i = 0;  
while (i < 5) {  
    console.log("Iterace:", i);  
    i++;  
}
```

- iii. **Cyklus do...while :**

- **Účel:** Spustí blok kódu jednou před kontrolou podmínky, pak opakuje, dokud je podmínka pravdivá.
- **Syntaxe:**

```
do {  
    // kód k vykonání  
} while (podmínka);
```

- **Příklad:**

```
let i = 0;  
do {  
    console.log("Iterace:", i);  
    i++;  
} while (i < 5);
```

iv. Cyklus **for...of** :

- **Účel:** Iterace přes iterovatelné objekty (pole, řetězce) a přístup k jejich hodnotám.
- **Syntaxe:**

```
for (const prvek of iterovatelné) {  
    // kód k vykonání  
}
```

- **Příklad:**

```
const fruits = ["Apple", "Banana", "Cherry"];  
for (const fruit of fruits) {  
    console.log("Ovoce:", fruit);  
}
```

v. Cyklus **for...in** :

- **Účel:** Iterace přes vlastnosti objektu.
- **Syntaxe:**

```
for (const klíč in objekt) {  
    // kód k vykonání  
}
```

- **Příklad:**

```
const user = { name: "Alice", age: 25, role: "Tester" };
for (const key in user) {
  console.log(key + ":", user[key]);
}
```

Použití v testovací automatizaci:

- **Testování s různými daty:** Iterace přes pole testovacích dat pro opakované testovací případy s různými vstupy.
- **Sériové operace:** Provedení posloupnosti testovacích kroků pod různými podmínkami nebo konfiguracemi.

D. Úvod do funkcí

Definice:

Funkce je znovupoužitelný blok kódu vykonávající specifický úkol. Umožňuje zabalit logiku pod jméno a to pak volat kdykoliv je potřeba—bez opakování stejného kódu.

Představte si to jako **recept**: jednou si ho napíšete a pak podle něj vaříte, aniž byste postup museli psát znovu.

- **Co jsou funkce?**
 - Znovupoužitelné bloky kódu určené k vykonání konkrétního úkolu.
 - Zvyšují modularitu a organizaci kódu.
- **Deklarace vs. výrazy funkcí:**
 - Rozdíly v syntaxi a chování při vyzdvihování (hoisting).
 - Kdy použít který typ.
- **Parametry a návratové hodnoty:**
 - Předávání dat do funkcí a získávání výsledků.
- **Rozsah platnosti (scope) a closures:**
 - Pochopení přístupnosti proměnných ve funkcích.
 - Úvod do closures a jejich významu.

Struktura funkce (syntaxe)

```
function functionName(parameter1, parameter2) {
  // Blok kódu
  return result;
}
```

Parametry a argumenty

- **Parametry** jsou zástupné názvy při definování funkce.
- **Argumenty** jsou skutečné hodnoty, které předáváte při volání funkce.

```
function greetUser(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greetUser("Anna"); // Výstup: Hello, Anna!
```

Návratové hodnoty

Funkce může poslat výsledek zpět pomocí klíčového slova `return`.

```
function multiply(a, b) {  
  return a * b;  
}  
  
let result = multiply(3, 4); // result = 12  
console.log(result);        // Výstup: 12
```

Pokud se `return` nepoužije, funkce vrací `undefined`.

Jednoduché příklady

✓ Sečtěte dvě čísla

```
function add(x, y) {  
  return x + y;  
}  
  
console.log(add(5, 7)); // Výstup: 12
```

✓ Zjistěte, zda je číslo sudé


```
function isEven(num) {  
  return num % 2 === 0;  
}
```

```
console.log(isEven(4)); // Výstup: true  
console.log(isEven(9)); // Výstup: false
```

✅ Zobrazte informace o uživateli

```
function showUserInfo(name, age) {  
  console.log(`Name: ${name}, Age: ${age}`);  
}
```

```
showUserInfo("Lucas", 29); // Výstup: Name: Lucas, Age: 29
```

Deklarace vs. výraz funkce

◆ Deklarace funkce

K dispozici **už před zápisem v kódu** (díky hoistingu):

```
sayHi();
```

```
function sayHi() {  
  console.log("Ahoj!");  
}
```

◆ Výraz funkce

Přiřazuje se do proměnné—**není vyzdvihován**, musí být nejdříve definován:

```
const sayBye = function() {  
  console.log("Sbohem!");  
};
```

```
sayBye();
```

Bonus: Šipkové funkce

Šipkové funkce jsou kratší způsob zápisu funkčních výrazů:

```
const greet = (name) => {  
  return `Hello, ${name}!`;  
};  
  
console.log(greet("Sarah"));
```

Scope a closures

Definice:

- **Scope:** Dostupnost proměnných a funkcí v různých částech kódu za běhu.
- **Closures:** Vlastnost JavaScriptu, kdy vnořená funkce má přístup k proměnným své vnější funkce, i po jejím skončení.

Podrobné vysvětlení:

A. Scope:

1. Globální scope:

- **Definice:** Proměnné deklarované mimo funkci nebo blok jsou v globálním rozsahu a dostupné kdekoli v kódu.
- **Příklad:**

```
var globalVar = "Jsem globální!";  
function displayGlobal() {  
  console.log(globalVar); // Dostupné  
}  
displayGlobal(); // Výstup: Jsem globální!  
console.log(globalVar); // Dostupné
```

2. Funkční scope:

- **Definice:** Proměnné deklarované v rámci funkce jsou přístupné pouze uvnitř této funkce a jejích vnořených funkcí.
- **Příklad:**

```
function outerFunction() {
  var functionVar = "Jsem uvnitř funkce!";
  function innerFunction() {
    console.log(functionVar); // Dostupné
  }
  innerFunction();
  console.log(functionVar); // Dostupné
}
outerFunction();
console.log(functionVar); // ReferenceError: functionVar není definována
```

3. Blokový scope:

- **Definice:** Proměnné deklarované v bloku ({}) pomocí `let` nebo `const` jsou dostupné pouze v tomto bloku.
- **Příklad:**

```
if (true) {
  let blockVar = "Jsem v bloku!";
  console.log(blockVar); // Dostupné
}
console.log(blockVar); // ReferenceError: blockVar není definována
```

B. Closures:

1. Definice:

- Closure vzniká tehdy, když vnořená funkce uchovává přístup k proměnným své vnější funkce i po jejím vykonání.

2. Příklad:

```
function outerFunction() {
  let outerVar = "Jsem z vnějšího rozsahu!";

  function innerFunction() {
    console.log(outerVar); // Přístup k outerVar
  }

  return innerFunction;
}

const myInnerFunction = outerFunction();
myInnerFunction(); // Výstup: Jsem z vnějšího rozsahu!
```

3. Použití v testovací automatizaci:

- **Privátní proměnné:** Zapouzdření proměnných, které nemají být dostupné globálně.
- **Továrny na funkce:** Tvorba specializovaných funkcí s přednastavenými parametry nebo chováním.

4. Výhody:

- **Ochrana dat:** Udržuje některé proměnné skryté před globálním rozsahem a zabraňuje jejich nechtěné změně.
- **Rozšířené možnosti:** Umožňuje tvorbu univerzálnějších a flexibilnějších funkcí, které si uchovávají stav napříč voláními.

Vizuální znázornění:

- **Schéma closure:** Ukazuje, jak si vnořená funkce uchovává přístup k proměnným vnější funkce i po jejím vykonání.

E. Doporučené zásady pro řídicí struktury a funkce

- **Čitelnost a udržovatelnost:**
 - Psát jasné a srozumitelné řídicí struktury.
 - Udržovat funkce zaměřené pouze na jeden úkol.
- **Vyhýbání se hlubokému vnořování:**
 - Strategie pro zamezení nadměrné složitosti a odsazování.
- **Pojmenovávací konvence:**
 - Výstižná jména funkcí a proměnných s jasným účelem.
- **Princip DRY (Don't Repeat Yourself):**
 - Snižování duplicit využíváním funkcí a cyklů.

Příklady kódu pro „Zásady nejlepší praxe pro řídicí struktury a funkce“

Zavádění těchto zásad zajistí, že váš kód bude čistý, udržovatelný a efektivní. Níže jsou uvedeny příklady prokládané doporučenými postupy v rámci řídicích struktur a funkcí.

A. Čitelnost a udržovatelnost

Špatná praxe: Hluboké vnořování

```
if (isUserLoggedIn) {  
  if (user.hasPermission) {  
    if (user.isActive) {  
      performSensitiveOperation();  
    }  
  }  
}
```

Dobrá praxe: Včasný návrat z funkce pro omezení vnořování

```
function performOperation(user) {  
  if (!user.isLoggedIn) return;  
  if (!user.hasPermission) return;  
  if (!user.isActive) return;  
  
  performSensitiveOperation();  
}
```

B. Funkce pouze s jedním úkolem

Špatná praxe: Funkce dělá víc věcí najednou

```
function processUserData(user) {  
  // Validace uživatele  
  if (!user.email) {  
    console.log("Neplatný uživatel");  
    return;  
  }  
  
  // Uložení uživatele do databáze  
  database.save(user);  
  
  // Odeslání uvítacího e-mailu  
  emailService.sendWelcomeEmail(user.email);  
}
```

Dobrá praxe: Oddělené funkce pro každý úkol

```

function validateUser(user) {
  if (!user.email) {
    console.log("Neplatný uživatel");
    return false;
  }
  return true;
}

function saveUser(user) {
  database.save(user);
}

function sendWelcomeEmail(user) {
  emailService.sendWelcomeEmail(user.email);
}

function processUserData(user) {
  if (!validateUser(user)) return;
  saveUser(user);
  sendWelcomeEmail(user);
}

```

C. Vyhýbání se hlubokému vnořování pomocí Guard Clauses

Špatná praxe: Víceúrovňové podmínky

```

function checkAccess(user) {
  if (user) {
    if (user.isActive) {
      if (user.hasAccess) {
        grantAccess();
      }
    }
  }
}

```

Dobrá praxe: Použijte Guard Clauses

```
function checkAccess(user) {  
  if (!user) return;  
  if (!user.isActive) return;  
  if (!user.hasAccess) return;  
  
  grantAccess();  
}
```

D. Použití popisných názvů

Špatná praxe: Nejasné názvy proměnných a funkcí

```
function doIt(a, b) {  
  if (a > b) {  
    return a;  
  } else {  
    return b;  
  }  
}
```

```
let x = doIt(5, 10);
```

Dobrá praxe: Výstižné názvy

```
function getHigherValue(firstValue, secondValue) {  
  if (firstValue > secondValue) {  
    return firstValue;  
  } else {  
    return secondValue;  
  }  
}
```

```
let higherScore = getHigherValue(5, 10);
```

F. Praktické příklady

- **Implementace podmíněné logiky v testech:**
 - Použití příkazů `if` pro různé scénáře testů.
- **Iterace přes testovací data:**
 - Použití cyklů pro procházení polí testovacích vstupů (data-driven testování).

- **Vytváření utilitních funkcí:**
 - Tvorba funkcí pro opakované kroky v testech pro zvýšení znovupoužitelnosti.

E. Jak řídicí struktury ovlivňují tok programu

Příkladový scénář: Autentizace uživatele

Představte si, že píšete testovací skript, který ověřuje autentizaci uživatele ve webové aplikaci. Řídicí struktury určují tok podle různých uživatelských vstupů a stavů systému.

Příklad kódu:


```

function authenticateUser(username, password) {
  if (!username || !password) {
    console.log("Uživatelské jméno i heslo jsou povinné.");
    return;
  }

  if (password.length < 6) {
    console.log("Heslo musí mít alespoň 6 znaků.");
    return;
  }

  // Simulace serverové autentizace
  let isAuthenticated = serverAuthenticate(username, password);

  if (isAuthenticated) {
    console.log("Uživatel úspěšně autentizován!");
    // Pokračovat v udělení přístupu
  } else {
    console.log("Autentizace selhala. Zkontrolujte přihlašovací údaje.");
    // Umožnit opakování/připomenutí hesla
  }
}

// Simulovaná serverová autentizace
function serverAuthenticate(username, password) {
  // Pro demonstraci - jakékoli heslo "password123" autentizuje úspěšně
  return password === "password123";
}

// Testovací případy
authenticateUser("testUser", "password123"); // Úspěšná autentizace
authenticateUser("testUser", "pass");        // Heslo je příliš krátké
authenticateUser("", "password123");          // Chybí uživatelské jméno
authenticateUser("testUser", "wrongPass");    // Autentizace selhala

```

Vysvětlení:

1. Úvodní kontroly:

- Používá příkazy `if` k ověření zadání uživatelského jména a hesla.
- Kontroluje minimální délku hesla.

2. Autentizační logika:

- Volá funkci `serverAuthenticate` pro simulaci validace na serveru.

- Výsledek vyhodnocuje dalším příkazem `if` .

3. Řízení toku:

- Podle podmínek směřuje program k různým blokům kódu tak, aby přístup získal pouze validní a autentizovaný uživatel.

Výsledek:

- Ukazuje, jak příkazy `if` a `else` ovlivňují běh programu dle různých podmínek a zajišťují robustní a bezpečný proces autentizace.

Ilustrace a reálné příklady

Ukázka, jak řídicí struktury ovlivňují tok programu

Příkladový scénář: Automatizované testování odeslání formuláře

Představte si, že píšete Cypress test pro automatizaci odeslání registračního formuláře. Řídicí struktury spravují různé testovací situace podle vstupů uživatele a odpovědí aplikace.

Příklad kódu:

```

describe('User Registration Form', () => {
  it('Submits the form with valid data', () => {
    cy.visit('/register');

    // Vyplňte formulář
    cy.get('#username').type('testUser');
    cy.get('#email').type('testuser@example.com');
    cy.get('#password').type('SecurePass123');

    // Podmíněná kontrola: Je tlačítko Odeslat povoleno?
    cy.get('#submit').then(($btn) => {
      if (!$btn.is(':disabled')) {
        cy.wrap($btn).click();
      } else {
        throw new Error('Tlačítko Odeslat je zakázáno');
      }
    });

    // Ověřte úspěšnou registraci
    cy.contains('Registration Successful!').should('be.visible');
  });

  it('Displays error with invalid email', () => {
    cy.visit('/register');

    // Vyplňte formulář s neplatným e-mailem
    cy.get('#username').type('testUser');
    cy.get('#email').type('invalid-email');
    cy.get('#password').type('SecurePass123');

    // Pokus o odeslání
    cy.get('#submit').click();

    // Podmíněná kontrola: Zobrazit chybovou zprávu při neplatném e-mailu
    cy.get('.error-message').then($msg => {
      if ($msg.is(':visible')) {
        cy.wrap($msg).should('contain', 'Invalid email address');
      } else {
        throw new Error('Chybová zpráva pro neplatný e-mail nebyla zobrazena');
      }
    });
  });
});

```

```
});  
});
```

Vysvětlení:

1. Podmíněné příkazy (`if`):

- Před kliknutím ověří, zda není tlačítko zakázané.
- Pokud je tlačítko povoleno, pokračuje; jinak vyhodí chybu.

2. Cykly:

- (V tomto příkladu nejsou přímo uvedeny, ale lze použít pro iteraci různých testovacích dat.)

3. Řízení toku:

- Podle vstupu (platný/neplatný e-mail) test kontroluje úspěch nebo příslušné chybové hlášení.

Výsledek:

- Ukazuje, jak příkazy `if` řídí spouštění testů podle různých stavů a zajišťují správnou reakci na různé vstupy a odpovědi aplikace.

2. Praktická část: cvičení a náměty na webové funkce

A. Cvičení s podmíněnými příkazy

• Cvičení:

- Napište funkci v JavaScriptu, která přijímá skóre uživatele a přiřadí známku dle předdefinovaných kritérií pomocí `if`, `else if` a `else`.
- Příklad:

```
function assignGrade(score) {  
  if (score >= 90) {  
    return 'A';  
  } else if (score >= 80) {  
    return 'B';  
  } else if (score >= 70) {  
    return 'C';  
  } else if (score >= 60) {  
    return 'D';  
  } else {  
    return 'F';  
  }  
}
```

- **Návrh na rozšíření do webové aplikace:**

- Vytvořte jednoduchý HTML formulář, do kterého mohou uživatelé zadat své skóre; JavaScriptová funkce pak po odeslání vypočítá a zobrazí odpovídající známku.

B. Cvičení s iterací pole

- **Cvičení:**

- Napište funkci v JavaScriptu, která přijímá pole čísel a vrátí nové pole obsahující pouze sudá čísla pomocí cyklu `for`.
- Příklad:

```
function filterEvenNumbers(numbers) {  
  let evenNumbers = [];  
  for (let i = 0; i < numbers.length; i++) {  
    if (numbers[i] % 2 === 0) {  
      evenNumbers.push(numbers[i]);  
    }  
  }  
  return evenNumbers;  
}
```

- **Návrh na webovou funkci:**

- Vytvořte webovou aplikaci, kde uživatelé zadají seznam čísel a aplikace zobrazí pole pouze sudých čísel podle definované funkce.

C. Cvičení s vytvářením a používáním funkcí

- **Cvičení:**

- Napište funkci v JavaScriptu, která přijímá dvě čísla jako parametry a vrátí jejich součet. Použijte tuto funkci v cyklu, který spočítá celkový součet pro pole dvojic čísel.
- Příklad:

```
function add(a, b) {  
  return a + b;  
}  
  
let pairs = [[1, 2], [3, 4], [5, 6]];  
let totalSum = 0;  
for (let i = 0; i < pairs.length; i++) {  
  totalSum += add(pairs[i][0], pairs[i][1]);  
}  
console.log(totalSum); // Výstup: 21
```

- **Návrh pro webovou aplikaci:**

- Umožněte uživatelům zadat několik dvojic čísel na stránce; aplikace spočítá a zobrazí celkový součet pomocí funkce `add` v cyklu.

D. Cvičení se scope a closures

- **Cvičení:**

- Demonstrujte koncept scope vytvořením funkce uvnitř jiné a ukažte, jak jsou proměnné dostupné.
- Příklad:

```
function outerFunction() {  
    let outerVariable = 'Jsem venku!';  
  
    function innerFunction() {  
        let innerVariable = 'Jsem uvnitř!';  
        console.log(outerVariable); // Přístupné  
        console.log(innerVariable); // Přístupné  
    }  
  
    innerFunction();  
    console.log(innerVariable); // ReferenceError: innerVariable není definována  
}  
  
outerFunction();
```

- **Návrh na webovou funkci:**

- Vytvořte interaktivní příklad na stránce, kde uživatelé uvidí, které proměnné jsou dostupné v různých rozsazích funkcí.

3. Možné otázky studentů

A. Řídicí struktury:

1. **Jaký je rozdíl mezi příkazy `if` a `switch` ?**

- **Odpověď:**

Oba příkazy slouží pro podmíněné vykonávání, avšak `if` je univerzálnější a zvládá libovolné i složitější podmínky/výrazy. Příkaz `switch` se lépe hodí, pokud řešíte více konkrétních hodnot jedné proměnné/výrazu.

2. Kdy použít cyklus `while` místo `for` ?

- **Odpověď:**

Použijte `while` , pokud není předem znám počet opakování a závisí na splnění určité podmínky. Cyklus `for` je vhodnější, pokud je počet iterací předem daný nebo snadno určitelný.

B. Funkce:

1. Jaký je rozdíl mezi deklarací a výrazem funkce?

- **Odpověď:**

Deklarace funkcí jsou „zdviženy“ (hoisted) – jsou načteny do paměti v průběhu kompilace a lze je volat ještě před jejich zápisem v kódu. Výrazy funkcí nejsou vyzdvihovány stejným způsobem a nelze je volat před jejich definicí.

2. Lze předat funkci jako argument jiné funkci?

- **Odpověď:**

Ano, v JavaScriptu jsou funkce objekty první třídy a lze je předávat jako argumenty, vracet je z funkcí nebo je přiřazovat do proměnných.

C. Cykly:

1. Jaký je rozdíl mezi cykly `for...of` a `for...in` ?

- **Odpověď:**

`for...of` se používá pro iteraci hodnot iterable objektů (pole, řetězce), zatímco `for...in` se používá pro iteraci vlastností objektu.

2. Jak zabránit nekonečné smyčce?

- **Odpověď:**

Ujistěte se, že podmínka cyklu bude někdy nesplněná. Dbejte na správu počítáčů a podmínek, abyste předešli situaci, kdy cyklus běží nekonečně.

D. Nejlepší praxe:

1. Proč je důležité, aby funkce měla pouze jeden účel?

- **Odpověď:**

Funkce vykonávající jeden úkol jsou přehlednější, lépe se testují, debuguje se a udržují. Zvyšují i znovupoužitelnost a snižují složitost každé funkce.

2. Co znamená DRY (Don't Repeat Yourself) v kontextu psaní funkcí?

- **Odpověď:**

DRY nabádá k odstranění duplicitního kódu tím, že opakující se části abstrahujete do funkcí. Výsledkem je přehlednější a lépe udržovatelný kód.

4. Doplnkový materiál: Doporučení

A. Oficiální dokumentace a příručky:

- Řídící struktury v JavaScriptu:
 - [MDN Control Flow](#)
- Funkce v JavaScriptu:
 - [MDN Functions](#)
- Cykly v JavaScriptu:
 - [MDN Loop Statements](#)

B. Tutoriály a články:

- Řídící struktury v JavaScriptu:
 - [W3Schools JavaScript Control Structures](#)
- Pochopení funkcí v JavaScriptu:
 - [FreeCodeCamp Functions](#)
- Cykly v JavaScriptu:
 - [JavaScript Loops Explained](#)

C. Interaktivní vzdělávací platformy:

- Codecademy:
 - [Learn JavaScript Control Flow](#)
- FreeCodeCamp:
 - [JavaScript Control Structures](#)
- [JavaScript.info](#):
 - [JavaScript Control Flow](#)

D. Videotutoriály:

- Traversy Media:
 - [JavaScript Control Flow Tutorial](#)
- The Net Ninja:
 - [JavaScript Functions Tutorial](#)
- Academind:
 - [JavaScript for Beginners: Functions and Control Flow](#)

E. Platformy na opakování/praktiku:

- **HackerRank:**
 - [JavaScript Control Structures Challenges](#)
- **LeetCode:**
 - [JavaScript Functions Problems](#)
- **Exercism:**
 - [JavaScript Track - Functions](#)

F. Komunity a podpora:

- **Stack Overflow:**
 - [JavaScript Control Structures](#)
 - [JavaScript Functions](#)
- **Reddit:**
 - [r/javascript](#)
- **Discord komunity:**
 - Připojte se na Discord servery zaměřené na JavaScript pro pomoc a diskusi.

5. Doporučené rozložení lekce na 3 hodiny

1. hodina: Úvod do řídicích struktur (60 minut)

- **Co jsou řídicí struktury? (15 minut):**
 - Přehled, jak řídicí struktury řídí tok vykonávání.
- **Podmíněné příkazy (30 minut):**
 - Vysvětlení `if`, `else if`, `else`, `switch`.
 - Praktické příklady relevantní pro testovací automatizaci.
- **Přestávka (5 minut)**

2. hodina: Cykly a funkce (60 minut)

- **Cykly (25 minut):**
 - Vysvětlení typů cyklů: `for`, `while`, `do...while`, `for...of`, `for...in`.
 - Příkladová použití, např. iterace dat v testech v Cypress.
- **Úvod do funkcí (25 minut):**
 - Deklarace vs. výrazy funkcí.
 - Parametry, návratové hodnoty, scope.

- Přestávka (5 minut)

3. hodina: Praktické cvičení a Q&A (60 minut)

- **Praktické úkoly (40 minut):**
 - **Podmíněné příkazy:**
 - Vytvoření funkce na přiřazení známky pomocí `if...else if...else`.
 - **Cykly:**
 - Napsání cyklu pro filtraci sudých čísel z pole.
 - **Funkce:**
 - Vývoj utilitní funkce pro opakující se kroky v testech.
- **Q&A (20 minut):**
 - Prostor pro dotazy studentů.
 - Dovyjasnění a upevnění klíčových pojmů lekce.

6. Další doporučení

A. Interaktivní ukázky:

- **Live coding:**
 - Ukázka psaní podmíněných příkazů a cyklů v reálném čase.
 - Ukázka definování a volání funkcí, včetně parametrů a scope.
- **Ladění pomocí `console.log`:**
 - Používání `console.log` v řídicích strukturách a funkcích pro sledování průběhu běhu programu a hodnot proměnných.

B. Poutavé vizuály:

- **Vývojové diagramy:**
 - Použít diagramy pro znázornění toků řízení programu.
- **Kódové úryvky:**
 - Zobrazovat přehledné a stručné příklady kódu na slajdech.
- **Schémata:**
 - Znázornit rozsah proměnných a funkcí.

C. Povzbuzujte zapojení:

- **Párové programování:**
 - Práce ve dvojicích na úlohách = lepší spolupráce a sdílení znalostí.

- **Rychlé ankety a kvízy:**
 - Průběžné ověřování znalostí a zajištění angažovanosti.

D. Jasně instrukce:

- **Postupné průvodce:**
 - Detailní pokyny pro každé praktické cvičení = všichni vše zvládnou.
- **Tipy pro řešení potíží:**
 - Upozorněte na časté chyby (například chyby syntaxe v cyklech a funkcích) a nabídněte řešení.

E. Podporující prostředí:

- **Povzbuzujte otázky:**
 - Otevřená atmosféra pro kladení dotazů.
- **Dávat víc příkladů:**
 - Nabídněte různé příklady pro každý koncept, aby si každý našel cestu ke správnému pochopení.